

세미나 2회차: ROS2 센서 데이터 처리

육시우

2026-06-22



목차

- 1. ROS2에서 카메라 이미지 데이터 처리
- 2. 거리 센서(LiDAR) 데이터
- 3. IMU 센서
- 4. 대회 설명

1-1. OpenCV란 ?

OpenCV(Open Source Computer Vision Library)는 컴퓨터가 인간처럼 시각 정보(사진, 동영상 등)를 이해하고 분석할 수 있도록 도와주는 **오픈소스 컴퓨터 비전 및 머신러닝 소프트웨어 라이브러리**

- **이미지 처리 및 변환:** 사진 크기 조절, 회전, 필터링(블러, 샤프닝), 색상 공간 변환(RGB ↔ HSV, Grayscale 등).
- **객체 검출 및 인식:** 이미지나 영상에서 사람의 얼굴, 번호판, 고양이 등을 찾아내고 추적.
- **특징점 추출 및 매칭:** 서로 다른 두 사진에서 같은 위치나 사물을 찾아내어 이어 붙이는 기술(파노라마 사진 만들기 등).
- **모션 분석 및 객체 추적:** CCTV 영상 등에서 움직이는 물체를 감지하고 동선을 따라가는 기능.
- **로봇 및 자율주행 기술:** 카메라 캘리브레이션을 통해 렌즈 왜곡을 보정하고, 주변 환경의 거리를 측정하거나 차선을 인식하는 기능.

1-2. OpenCV란 ?

→ 실제 활용 방법: 머신러닝/딥러닝 프레임워크(TensorFlow, PyTorch)나 객체 탐지 모델(YOLO)과 함께 사용하면, 카메라로 입력받은 영상을 OpenCV로 전처리한 뒤 AI 모델로 던져주는 형태로 사용

1-2. ROS2 이미지 데이터 구조

ROS2에서 카메라 영상은

`sensor_msgs/msg/Image` 형식으로 전송됩니다.

필드	설명
<code>header</code>	시간·좌표 프레임 정보
<code>height</code> , <code>width</code>	이미지 크기
<code>encoding</code>	영상 포맷 (예: bgr8, mono8 등)
<code>data</code>	실제 픽셀 데이터

→ OpenCV의 cv2 이미지와 형식이 다르므로 **cv_bridge** 패키지를 이용해 변환 필요함

1-3. 실습

```
sudo apt install ros-humble-cv-bridge ros-humble-image-transport python3-opencv  
python3 -c "import cv2; print(cv2.__version__)"
```

→ 패키지 설치 후 확인 과정

```
mkdir -p ~/ros2_ws/src
```

→ 디렉토리 생성

```
cd ~/ros2_ws/src  
ros2 pkg create --build-type ament_python camera_subscriber_pkg
```

→ 디렉토리로 들어가서 카메라 노드용 패키지 생성

1-3. 실습

```
cd ~/ros2_ws/src/camera_subscriber_pkg/camera_subscriber_pkg  
nano camera_subscriber.py
```

→ 노드 파일 생성

1-3. 실습

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2

class CameraSubscriber(Node):

    def __init__(self):
        super().__init__('camera_subscriber')
        self.subscription = self.create_subscription(
            Image, '/camera/image_raw', self.listener_callback, 10)
        self.subscription
        self.br = CvBridge()
        self.get_logger().info('CameraSubscriber node started.')

    def listener_callback(self, data):
        try:
            # ROS Image → OpenCV 이미지로 변환
            current_frame = self.br.imgmsg_to_cv2(data, "bgr8")

            # 화면에 영상 표시
            cv2.imshow("Camera View", current_frame)
            cv2.waitKey(1)

        except Exception as e:
            self.get_logger().error(f'Error converting image: {e}')

def main(args=None):
    rclpy.init(args=args)
    node = CameraSubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    cv2.destroyAllWindows()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```


1-3. 실습

```
entry_points={
    'console_scripts': [
        'camera_subscriber = camera_subscriber_pkg.camera_subscriber:main',
    ],
},
```

→ `setup.py` 파일에서 `entry_point`를 찾아 위와 같이 수정: `camera_subscriber.py`이 터미널에 실행할 수 있는 노드라는 것을 알려주는 작업

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

→ 빌드

1-3. 실습

▶ 1 카메라 퍼블리셔 실행

ROS2에는 테스트용 가상 카메라 퍼블리셔가 포함되어 있습니다.

```
ros2 run image_tools cam2image
```

(실제 웹캠을 사용할 경우, `/dev/video0` 디바이스에서 자동 인식됩니다.)

▶ 2 영상 구독 노드 실행

새 터미널에서 다음 명령 실행:

```
ros2 run camera_subscriber_pkg camera_subscriber
```

💡 실행 결과

- 새로운 창에 실시간 영상이 표시됩니다.
- 콘솔에 다음과 같은 로그가 출력됩니다:

```
[INFO] [camera_subscriber]: CameraSubscriber node started.
```

1-3. 실습

```
admin2@admin2: ~/ros2_ws

admin2@admin2: ~/ros2_ws 77x23
[INFO] [1782119753.076065673] [cam2image]: Publishing image #193
[INFO] [1782119753.108075302] [cam2image]: Publishing image #194
[INFO] [1782119753.140048054] [cam2image]: Publishing image #195
[INFO] [1782119753.176056053] [cam2image]: Publishing image #196
[INFO] [1782119753.207930913] [cam2image]: Publishing image #197
[INFO] [1782119753.240047248] [cam2image]: Publishing image #198
[INFO] [1782119753.276037439] [cam2image]: Publishing image #199
[INFO] [1782119753.308082692] [cam2image]: Publishing image #200
[INFO] [1782119753.344083142] [cam2image]: Publishing image #201
[INFO] [1782119753.375949502] [cam2image]: Publishing image #202
[INFO] [1782119753.407974344] [cam2image]: Publishing image #203
[INFO] [1782119753.444135485] [cam2image]: Publishing image #204
[INFO] [1782119753.475893298] [cam2image]: Publishing image #205
[INFO] [1782119753.511962851] [cam2image]: Publishing image #206
[INFO] [1782119753.544008551] [cam2image]: Publishing image #207
[INFO] [1782119753.576094694] [cam2image]: Publishing image #208
[INFO] [1782119753.611971293] [cam2image]: Publishing image #209
[INFO] [1782119753.644086157] [cam2image]: Publishing image #210
[INFO] [1782119753.680020615] [cam2image]: Publishing image #211
[INFO] [1782119753.712081783] [cam2image]: Publishing image #212
[INFO] [1782119753.744069151] [cam2image]: Publishing image #213
[INFO] [1782119753.780137735] [cam2image]: Publishing image #214
[INFO] [1782119753.812050017] [cam2image]: Publishing image #215
[INFO] [1782119753.847974097] [cam2image]: Publishing image #216

admin2@admin2: ~/ros2_ws 37x23
hon3.10/dist-packages/rclpy/__init__.py", line 130, in shutdown
    _shutdown(context=context)
    File "/opt/ros/humble/local/lib/pyth
hon3.10/dist-packages/rclpy/utilities
.py", line 58, in shutdown
    return context.shutdown()
    File "/opt/ros/humble/local/lib/pyth
hon3.10/dist-packages/rclpy/context.p
y", line 102, in shutdown
    self.__context.shutdown()
rclpy._rclpy_pybind11.RCLError: failed
to shutdown: rcl_shutdown already c
alled on the given context, at ./src/
rcl/init.c:241
[ros2run]: Process exited with failur
e 1
admin2@admin2:~/ros2_ws$ export ROS_D
OMAIN_ID=0
admin2@admin2:~/ros2_ws$ ros2 run cam
era_subscriber_pkg camera_subscriber
[INFO] [1782119609.563386040] [camera
_subscriber_node]: 카메라 노드가 정상
시작되었습니다.
```

→ 원래 영상도 나와야 하는데 원인을 못 찾았다. 노트북의 문제일 수도 있고
카메라랑 연동이 잘 안되었을 수도 있다.

2-1. LiDAR란?

- LiDAR(Light Detection and Ranging)는 레이저를 발사해 반사된 신호로 거리를 측정하는 센서
- ROS2 LaserScan 데이터 구조

필드	설명
<code>angle_min</code>	시작 각도 (라디안)
<code>angle_max</code>	종료 각도
<code>angle_increment</code>	스캔 간격
<code>ranges</code>	거리 데이터 배열
<code>intensities</code>	반사 강도 (선택적)

→ 결과값은 LiDAR에서 수신한 거리 데이터가 극좌표 그래프로 표시됨.

2-2. 확장 실습

```
min_distance = np.min(ranges)
if min_distance < 0.5:
    self.get_logger().warn(f"⚠ Obstacle too close! Distance = {min_distance:.2f} m")
```

`min_distance = np.min(ranges) →`

`ranges`는 라이다 센서가 360도 전 방향으로 쏜 수백 개의 레이저 거리 값들이 담긴 배열(리스트)이다.

`np.min(ranges)`: NumPy 라이브러리를 사용해 수많은 거리 값 중 가장 짧은 거리 (가장 가까운 장애물까지의 거리)를 하나 찾아내서 `min_distance`라는 변수에 저장하는 것.

2-2. 확장 실습

```
min_distance = np.min(ranges)
if min_distance < 0.5:
    self.get_logger().warn(f"⚠ Obstacle too close! Distance = {min_distance:.2f} m")
```

if min_distance < 0.5 → 방금 찾은 가장 가까운 사물과의 거리가 0.5보다 작아지면 아래의 경고 코드를 실행하라는 뜻

→ 실제 프로젝트에서는 위 조건이 만족될 때 로봇의 속도를 제어하는 토픽을 강제로 조정해서 급정거 시키는 방식으로 활용할 수 있다.

3. IMU란?

- IMU(Inertial Measurement Unit, 관성 측정 장치)는 로봇이 **기울기, 회전, 가속도** 등을 감지하는 센서이다

데이터 종류	단위	설명
선형 가속도	m/s^2	x, y, z 방향의 가속도
각속도	rad/s	회전 속도
자세(orientation)	quaternion	로봇의 방향을 표현

3. ROS2 IMU 데이터 구조

필드	타입	설명
<code>orientation</code>	<code>geometry_msgs/Quaternion</code>	자세(회전)
<code>angular_velocity</code>	<code>geometry_msgs/Vector3</code>	각속도
<code>linear_acceleration</code>	<code>geometry_msgs/Vector3</code>	선형 가속도

→ 실제로 센서를 달고 시각화를 시키면 선형 가속도와 각속도의 그래프 창이 따로 표시됨. 이를 통해 로봇의 움직임을 데이터화하여 확인할 수 있음.

4. 대회 설명

HL FMA(Future Mobility Award) 2026 참가 신청서

HL FMA(Future Mobility Award) 2026
자율주행 경진대회를 개최합니다.

- 주최 : HL만도, HL클레무브, 한국도로교통공단
- 주관 : 한라대학교 SW중심대학사업단

1. 모집부문

1) HL FMA[1/5]

- 모집대상 : 전국 대학생 (팀당 6인 이하)
- 인공지능을 이용한 자율주행 자동차 경진대회

2) HL FMA[VLf]

- 모집대상 : 전국 고등학생 (팀당 3인 이하)
- 라즈베리파이를 이용한 자율주행 자동차 경진대회

3) HL FMA[시뮬레이션]

- 모집대상 : 전국 대학생 (팀당 3인 이하)
- 자율주행 시뮬레이터를 이용한 자율주행 경진대회

※ 인원수에 맞춰 작성 / 팀원이 없는 경우에는 "없음"으로 작성 후 제출

2.운영 일정

- 1) 참가접수 : 26.6.4.(목) ~ 7.12.(일)
- 2) 대회설명회 : 2026.7.28.(화) 14:00~15:00 / 온라인 진행
- 3) 경진대회 일정 및 장소
 - HL FMA [1/5] : 26.9.5.(토) ~ 6.(일), 26.9.19.(토) ~ 20.(일) / 용인운전면허시험장
 - HL FMA [VLf] : 26.9.12.(토) ~ 9.13.(일) / (재)원주미래산업진흥원
 - HL FMA [시뮬레이션] : 26.9.12.(토) / (재)원주미래산업진흥원

※ 운영 일정은 상황에 따라서 변경될 수 있습니다.

문의 : 한라대학교 SW중심대학사업단 hl.fma@halla.ac.kr

4. 대회 설명

대회 일정

- 자율연습 —————
2026.9.5.(토) ~ 9.6.(일)

- 예선(기능점검) 및 자율연습 —————
2026.9.19.(토)

- 본선 —————
2026.9.20.(일)

4. 대회 설명

☆ 평가기준

- 자동차 운전면허시험 기준에 준하며, 특정 미션이 추가됨

☆ 차량 제원 및 규정안내

- 1/5 스케일 플랫폼 기준 표준 제원입니다. 아래 허용 범위 내 차량만 출전할 수 있습니다.

스케일	1/5 스케일 자율주행 플랫폼
전장 / 전폭	1500mm 이내 / 800mm 이내
전고	550mm 이내 (센서 포함)
구동방식	전동(BLDC/브러시) 모터 구동, 전·후륜 조향 가능
구동 전압	24V 이하 (※ 전원 규정 필수 준수)
센서 구성	LiDAR·카메라·IMU·초음파 등 종류·개수 자유[신품기준 300만원 이하]
연산 장치	Jetson · 임베디드 보드 · MCU 등 자유 구성

※ 1/5 플랫폼 일반 표준 기준. 최종 규격은 운영 본부 배포 규정·도면을 우선 적용하며, 규격 범위 내 차종·센서 종류/개수는 자유.

4. 대회 설명

☆ 사용 가능 차량 모델 3종

• 참가팀은 아래 3개 모델 중 1대를 선택해 출전합니다. 어떤 모델을 사용하든 제원·전원·센서 규정을 동일하게 적용받습니다.

T870 / T8 sports

01



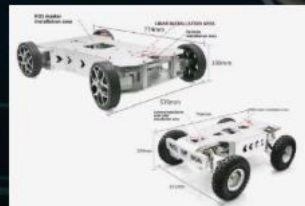
F8

02



기타

03



사용 가능 모델은 위 3종으로 한정하며, 3종 중 1대를 선택해 출전합니다.
단, 어떤 모델을 사용하더라도 차량 전압은 24V로 제한됩니다.

4. 대회 설명

☆ 참가차량 점검 - 참가자 상호 점검

- 운영 본부의 일방 점검이 아닌, 참가자 상호 점검으로 진행합니다. 상대 팀과 서로의 차량을 점검해 제원·공정성 심의를 선수(팀)간에 직접 수행합니다.

참가차량의 점검은 참가자 상호 점검을 실시함. 참가차량의 센서를 상대 팀 상호간에 점검을 통하여 제원 및 공정성에 대한 심의를 선수간(팀)에 진행하여 공정성을 확보하고자 함.



경진대회에 사용되는
운영체제 및 개발언어
는
제한이 없음



버스운전면허 시험장을
전체 지정시간 내
완주하여야 함



우천시 차량이
주행할 수 있는
방안 마련 필요



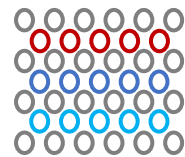
자율주행 차량
운영 계획서 제출
(별도공지)



자세한 규정은
네이버 카페에
공시 예정

Q & A

Thank you for your attention



Architecture and Compiler
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU
ACE Lab. (hw051656@gmail.com)